LINEAR MODEL SELECTION & REGULARIZATION

Ridge Regression, the Lasso & Elastic Net

Elsa Ferreira Gomes

2026 / 2027

Three sources of generalization error

A model's generalization error is the sum of three very different errors:

Bias

Wrong assumptions (e.g. assuming linear when it's quadratic).
High bias → underfits.

Variance

Excessive sensitivity to small changes in training data.
High variance → overfits.

Irreducible error

Noise in the data itself.
Only fixable by cleaning the data (e.g. removing outliers).

Increasing complexity: ↑ variance, ↓ bias. **Reducing complexity:** ↑ bias, ↓ variance.

Regularizing linear models

Overfitting

Performs well on training data, poorly on unseen (test) data.

Underfitting

Performs poorly on both training and test data.

Regularization avoids overfitting, especially when there's a large gap between train and test performance.

The number of features (used in training) stays the same, but the **magnitude of the coefficients is reduced.**

RECAP

Recall: the linear model

Training a linear model means finding the coefficients that minimize RSS:

$$Y = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$$

$$RSS = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} \left(f(x_i) - \hat{f}(x_i) \right)^2 = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2$$

*RSS is the **Loss Function** that **ordinary least squares minimizes***
- everything in this lecture is about adding a second term to this loss.

MOTIVATION

Least squares: the need for alternatives

Prediction accuracy

When $p > n$, there's no longer a unique least-squares estimate: variance becomes infinite!

Even when n isn't much bigger than p, **variance can be high**.

Model interpretability

Removing irrelevant features (setting their coefficients to zero) gives a model that's easier to interpret.

THE TOOLKIT

Three classes of methods

1

Subset Selection

- Identify a subset of the p predictors believed relevant
- fit least squares on that reduced set.

2

Regularization (Shrinkage)

- Fit a model using all p predictors but shrink the coefficients towards zero.
- Reduces variance, can also select variables.

3

Dimension Reduction

- Project the p predictors into an M-dimensional subspace ($M < p$),
- then fit least squares on those M projections.

Today's focus: Regularization specifically Ridge, the Lasso, and Elastic Net.

Ridge Regression and the Lasso

- Subset selection uses least squares on a subset of the predictors
- As an alternative, we can fit a model with all *p* predictors, but constrain or regularize the coefficient estimates
- Equivalently: shrink the coefficient estimates towards zero
- It may not be obvious why this helps, but **shrinking can significantly reduce variance**

Ridge Regression: short introduction

- A variation of linear regression, designed to address **multicollinearity**
- **Multicollinearity**: two or more predictors are moderately or highly correlated with one another
- When predictors are **highly correlated**, ordinary least squares can become unstable, with unreliable estimates
- **Ridge** introduces a **regularization term** (the **L2 penalty**) that **penalizes large coefficients**, stabilizing the model and preventing overfitting

What Ridge actually minimizes

The Ridge estimates β^R are the values that minimize:

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2=RSS+\lambda\sum_{j=1}^{p}\beta_j^2$$

$\lambda \geq 0$ is a **tuning parameter**, determined separately.

Remember: ordinary least squares estimates $\beta_0,\ldots,\beta_p$ by minimizing just the Residual Sum of Squares (RSS) - the first term here.

The shrinkage penalty, and the role of λ

Value of λ	Effect
$\lambda = 0$	No penalty: Ridge reduces exactly to ordinary least squares
$\lambda \to \infty$	Penalty dominates: coefficients shrink all the way towards zero
$0 < \lambda < \infty$	Controls the relative impact of RSS vs. the shrinkage penalty

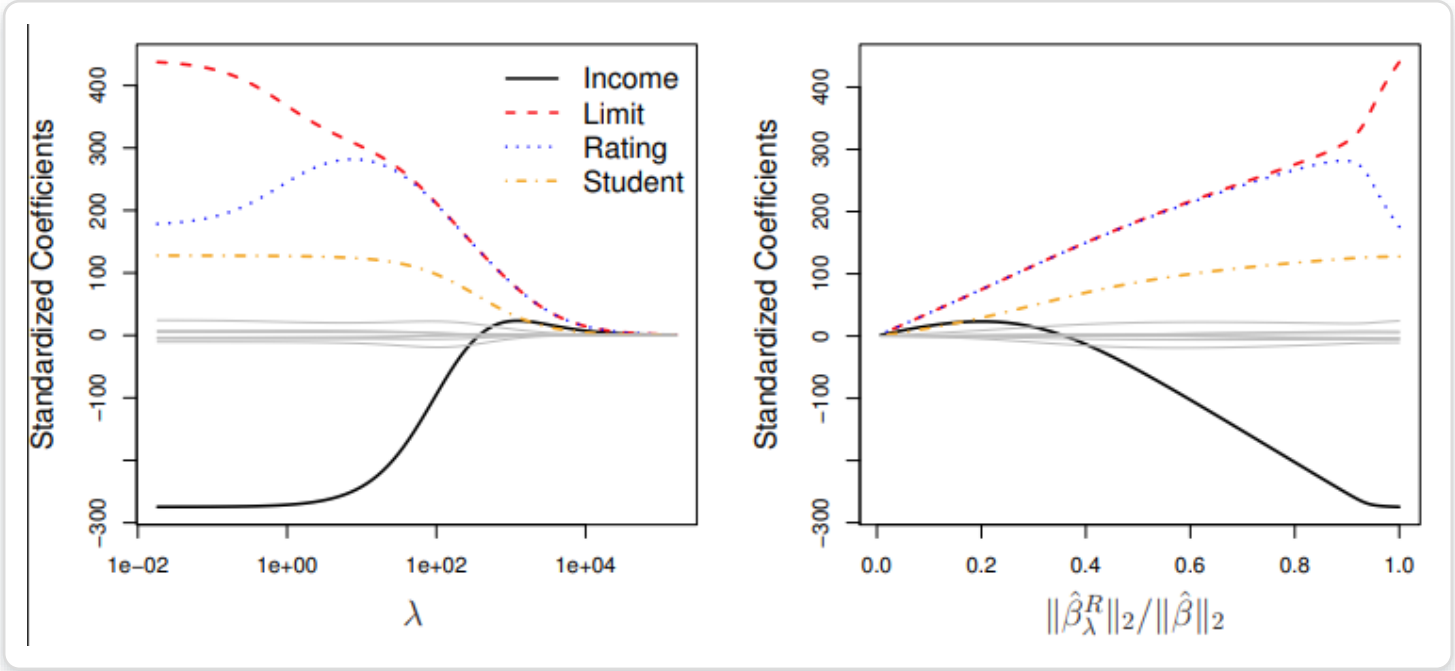
The **penalty $\lambda\Sigma\beta_j^2$** is **small** when the **coefficients are close to zero**

- it has the effect of shrinking them towards zero.

Selecting a good λ is critical

- cross-validation is the standard way to choose it.

Coefficient paths, as λ varies

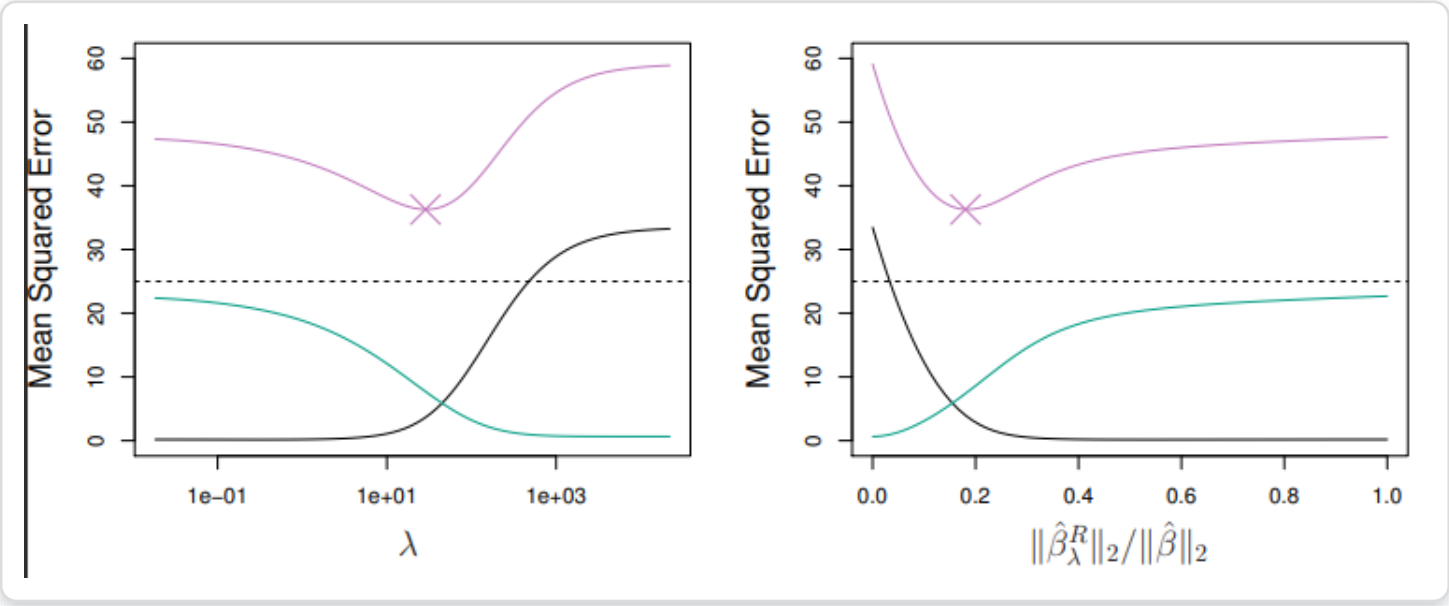


Left: each curve is one variable's Ridge estimate, as a function of λ.

Right: the same coefficients, now plotted against $\|\beta^R\lambda\|_2 / \|\beta\|_2$
- the fraction of the least-squares L2 norm retained.

Ridge: the bias-variance trade-off

Simulated data: n=50 observations, p=45 predictors, all with nonzero coefficients:



Squared bias (black), **variance** (green), test MSE (purple). **Purple crosses** mark the λ with smallest MSE (dashed lines show the minimum possible MSE).

CASE STUDY

Predicting Baseball Salaries

The **Hitters** dataset: 263 players, 19 predictors (batting stats, career stats, team) and predicting **Salary**.

We'll fit Ridge, Lasso, and Elastic Net, and compare them honestly.

CASE STUDY · DATA

The Hitters dataset

Major League Baseball statistics, used to predict each player's Salary:

Group	Variables
Recent-season stats	AtBat(At bats), Hits, HmRun(Home runs), Runs(Runs scored),RBI(Runs Batted In), Walks(Bases on balls)
Career totals ("C" prefix)	CAtBat, CHits, CHmRun, CRuns, CRBI, CWalks, Years
Defensive stats	PutOuts, Assists, Errors
Categorical	League, Division, NewLeague
Target	Salary (annual salary)

263 observations (59 rows with missing Salary dropped from the original 322) · **19 predictors** + 1 response (Salary) = 20 columns

CASE STUDY · DATA

Why this dataset suits Ridge & Lasso

Multicollinearity, by design: many career-total variables move together;

- A player with high CHits also tends to have high CRuns and CRBI.
- They're all really measuring the same underlying thing: “how long, and how good.”

Ridge's job

Stabilizes estimates when predictors are this correlated
- exactly the scenario Ridge was designed for.

Lasso's job

Among redundant, correlated variables,
- zero out the ones that add no new information once the others are in.

We can see this exactly: *Lasso zeroed out CAtBat and Chits; redundant once CRuns was already in the model.*

Loading & preparing the data

PYTHON

```
df = pd.read_csv('Hitters.csv').dropna()
df.info()

X = df.drop(columns=["Salary"])
y = df["Salary"]

X = pd.get_dummies(X, drop_first=True)

X_train,X_test,y_train,y_test = train_test_split(X, y,test_size=0.2,random_state=1)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

OUTPUT

```
263 entries, 20 columns
(after dropping missing
 Salary values)

19 predictors after
one-hot encoding
(League, Division,
NewLeague)
```

***Standardizing** (**scaling**) predictors first is essential for Ridge/Lasso - the **penalty term is sensitive to the scale of each variable**.*

Fitting Ridge across a range of λ *(or alpha argument of function Ridge())*

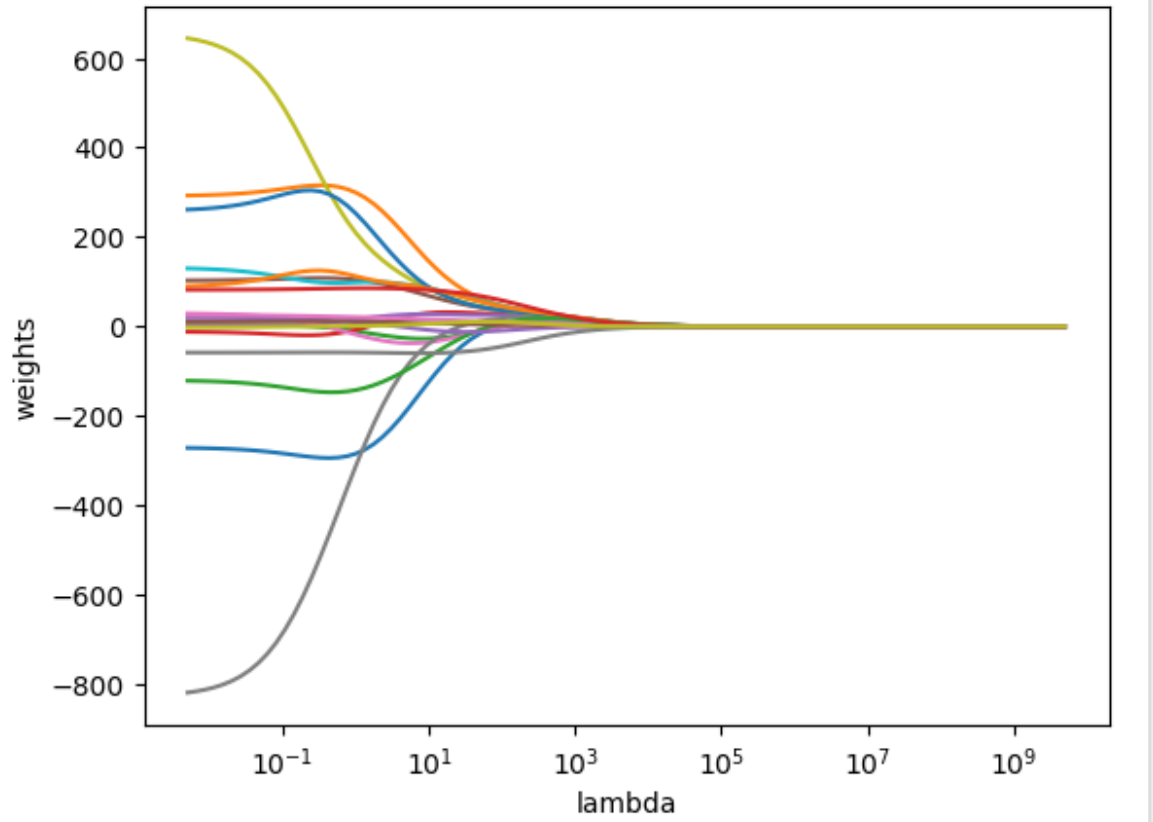
PYTHON

```
alphas = 10**np.linspace(10,-2,100)*0.5

ridge = Ridge()
coefs = []

for a in alphas:
    ridge.set_params(alpha=a)
    ridge.fit(X_train, y_train)
    coefs.append(ridge.coef_)

ax.plot(alphas, coefs)
ax.set_xscale('log')
```



[Ridge coefficients as a function of the L2 Regularization — scikit-learn 1.9.1 documentation](https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html);
https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html

A concrete choice: λ = 4

PYTHON

```
ridge2 = Ridge(alpha=4)
ridge2.fit(X_train, y_train)
pred2 = ridge2.predict(X_test)

mean_squared_error(y_test, pred2)
```

TEST MSE

126,497.6

A moderate penalty: coefficients are shrunk, but every predictor still contributes
- ***Ridge never zeroes anything out exactly**.*

What if λ is enormous?

PYTHON

```
ridge3 = Ridge(alpha=10**10)
ridge3.fit(X_train, y_train)
pred3 = ridge3.predict(X_test)

mean_squared_error(y_test, pred3)
```

TEST MSE

229,163.2

Much worse than λ=4! Every coefficient is squashed to ≈0 (essentially the null model). Over-shrinking adds bias faster than it removes variance.

And at the other extreme: λ = 0

PYTHON

```
ridge2 = Ridge(alpha=0)   # = ordinary least squares
ridge2.fit(X_train, y_train)
pred = ridge2.predict(X_test)

mean_squared_error(y_test, pred)
```

TEST MSE

133,595.0

Higher than λ=4's 126,497.6!

This confirms Ridge regression with a well-chosen λ genuinely outperforms plain least squares here.

CASE STUDY · RIDGE

Three values of λ, side by side

λ	Test MSE	What's happening
0 (= OLS)	133,595.0	No shrinkage at all
4	126,497.6	Best of these three — meaningful shrinkage
10^{10} (huge)	229,163.2	Over-shrunk — essentially the null model

The pattern: both extremes (no shrinkage, too much shrinkage) perform worse than a well-chosen middle value. This U-shape in test error, as a function of λ, is exactly the bias-variance trade-off from slide 2, made concrete.

Choosing λ properly: RidgeCV

Instead of arbitrarily guessing λ values, let cross-validation choose:

PYTHON

```
from sklearn.linear_model import RidgeCV

ridgecv = RidgeCV(alphas=alphas,
    scoring='neg_mean_squared_error')
ridgecv.fit(X_train, y_train)
ridgecv.alpha_

ridge4 = Ridge(alpha=ridgecv.alpha_)
ridge4.fit(X_train, y_train)
mean_squared_error(y_test,
    ridge4.predict(X_test))
```

BEST λ

2.32

TEST MSE

126,176.9

Our best manual guess (λ=4) was already close!

Lasso: overcoming Ridge's disadvantage

Ridge's disadvantage: unlike subset selection, Ridge will always include all ***p*** predictors in the final model
- just with small coefficients, **never exactly zero**.

- The **Lasso** overcomes this using the **L1 norm** instead of L2
- Forces some **coefficients** to be **exactly zero**, when λ is large enough
- Like best subset selection, the **Lasso performs variable selection**
- Yields sparse models (involving only a subset of the variables)

What the Lasso actually minimizes

The Lasso estimates β are the values that minimize:

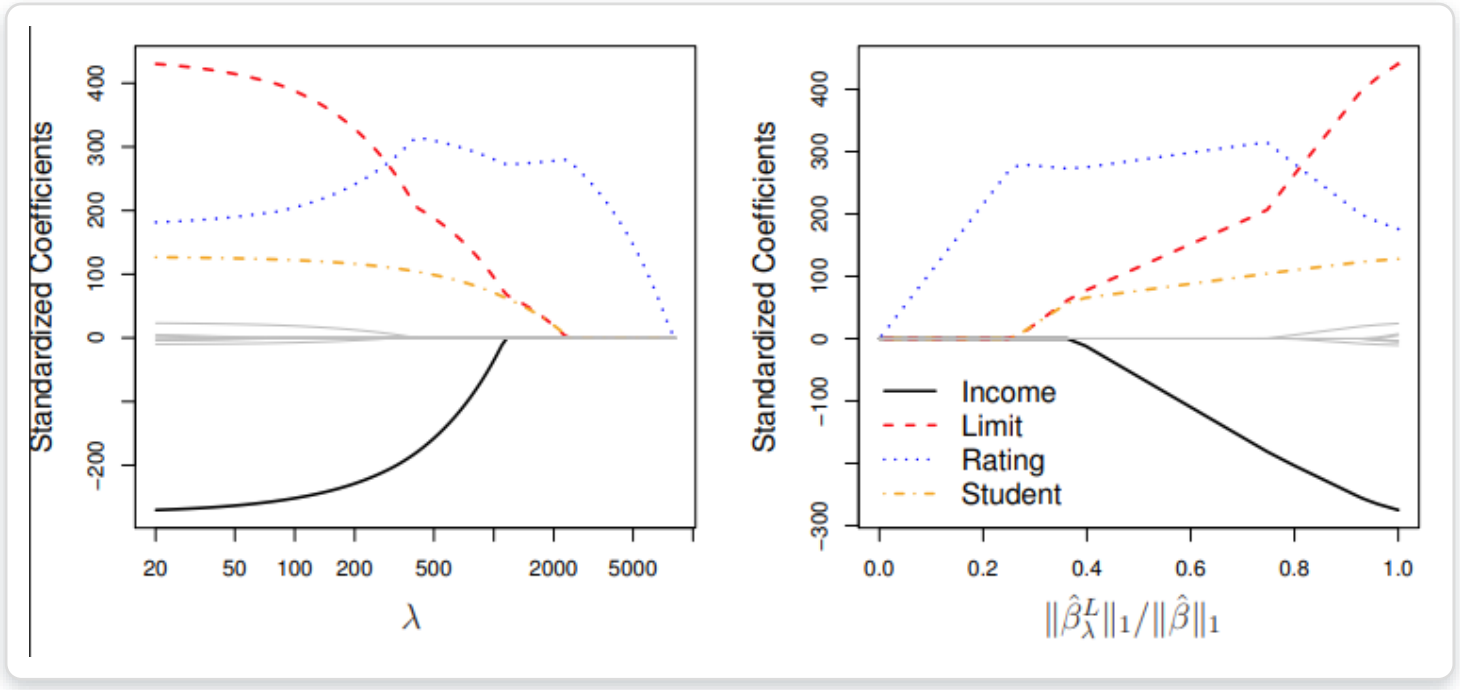
$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}|\beta_j|=RSS+\lambda\sum_{j=1}^{p}|\beta_j|$$

The only change from Ridge: `L1 norm` ($\|\beta\|_1 = \Sigma|\beta_j|$) instead of L2 norm.

This single change in the penalty's shape is precisely what **allows coefficients to reach exactly zero**, as we'll see visually next.

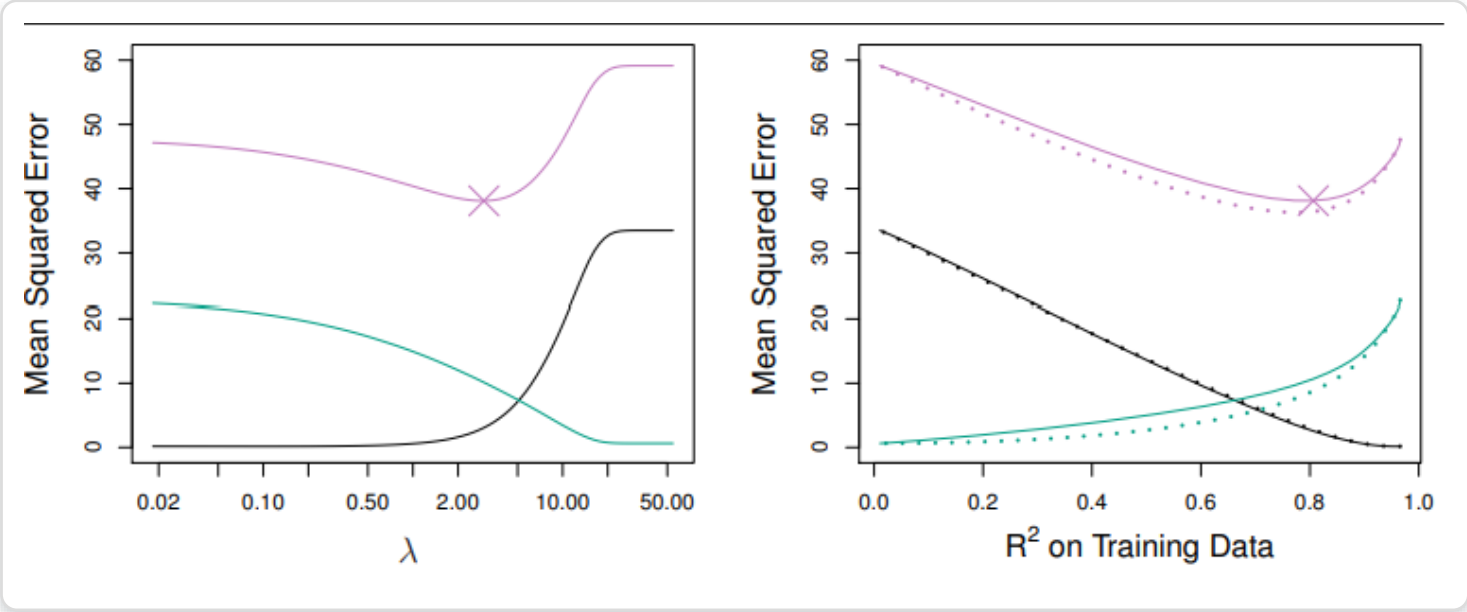
Lasso — scikit-learn 1.9.1 documentation

Lasso coefficient paths



Notice how curves don't just shrink towards zero: ***they actually hit exactly zero****, at different values of the tuning parameter, and stay there.*

Lasso vs. Ridge: bias, variance & MSE



Left: squared bias (black), variance (green), test MSE (purple) for the Lasso.

Right: Lasso (solid) vs. Ridge (dashed), plotted against training R^2. Crosses mark each method's minimum-MSE point.

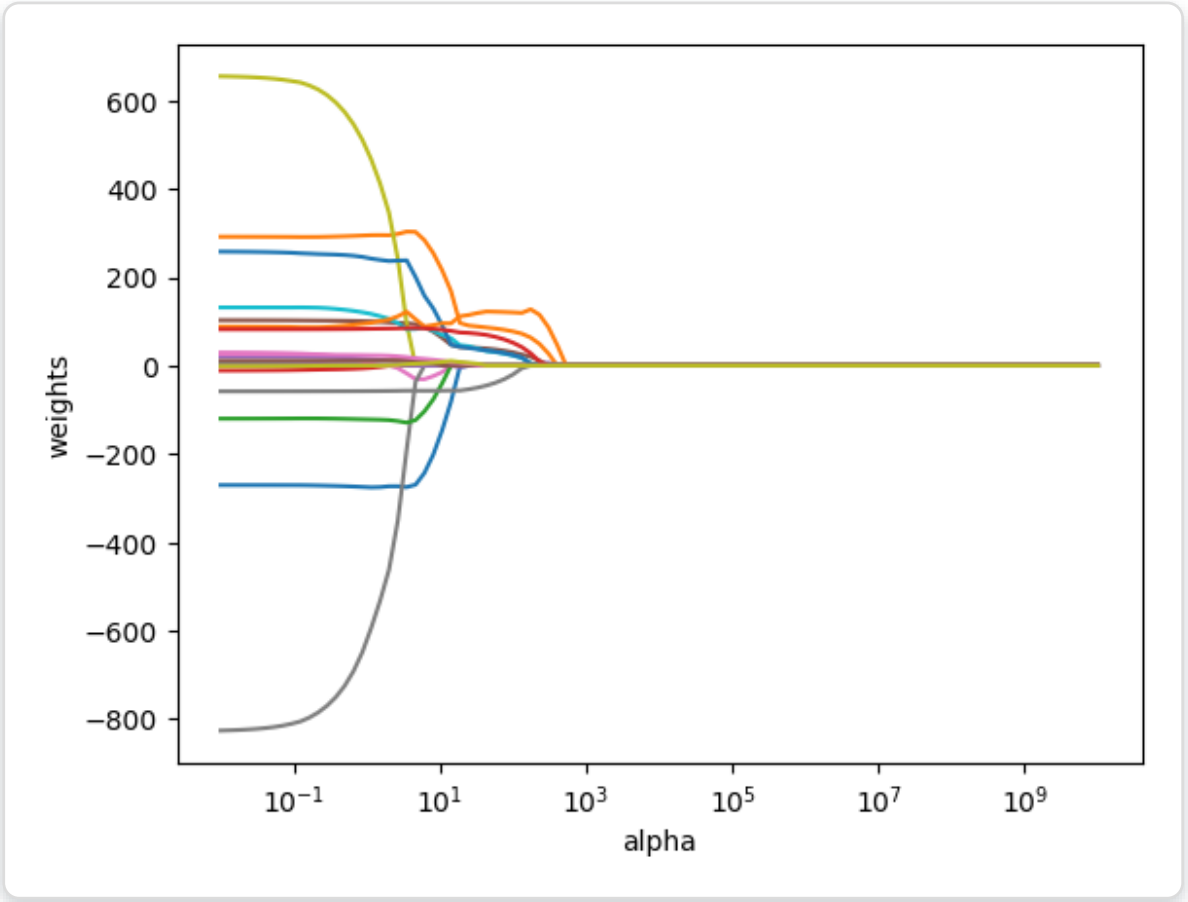
Lasso coefficient paths, on Hitters

PYTHON

```
lasso = Lasso(max_iter=10000)
coefs = []

for a in alphas:
    lasso.set_params(alpha=a)
    lasso.fit(scale(X_train), y_train)
    coefs.append(lasso.coef_)

ax.plot(alphas*2, coefs)
ax.set_xscale('log')
```



Depending on λ, some coefficients are exactly zero (real variable selection, on real data).

Choosing λ with LassoCV

PYTHON

```
lassocv = LassoCV(cv=10, max_iter=1000)
lassocv.fit(X_train, y_train)

lasso.set_params(alpha=lassocv.alpha_)
lasso.fit(X_train, y_train)

mean_squared_error(y_test,
    lasso.predict(X_test))
```

TEST MSE (Lasso, CV-tuned)

124,887.9

Lasso's real advantage: sparsity

Of the 19 coefficients, 6 are now exactly zero: a genuinely simpler, more interpretable model:

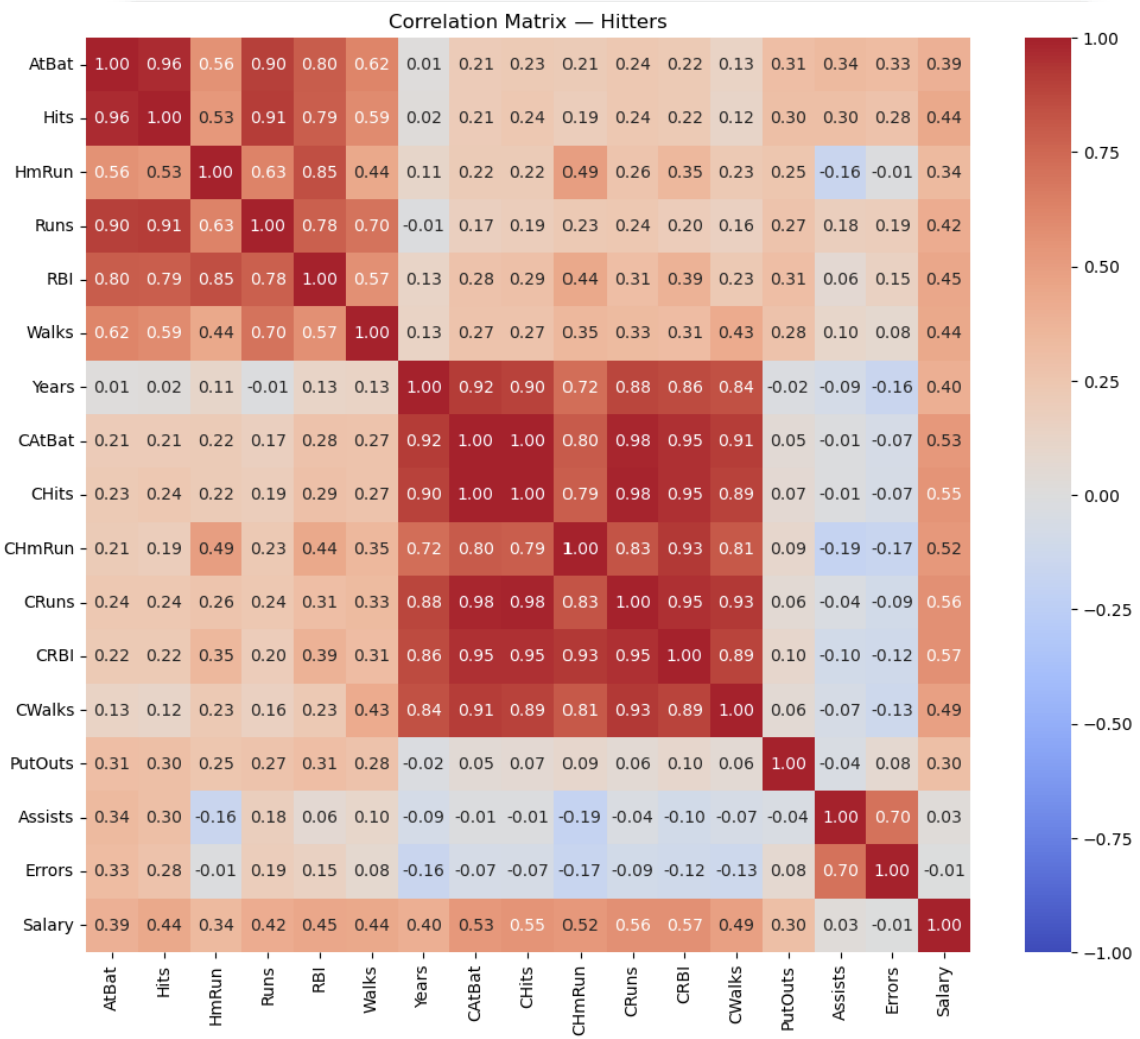
Variable	Coef.
AtBat	-234.21
Hits	277.64
HmRun	0.00
Runs	0.00
RBI	0.00
Walks	85.07

Variable	Coef.
Years	-29.32
CAtBat	0.00
CHits	0.00
CRuns	151.85
Assists	0.00
PutOuts	83.94

Zeroed-out variables (HmRun, Runs, RBI, CAtBat, CHits, Assists) are judged uninformative once the others are in the model
Lasso has quietly done feature selection for you.

The correlation matrix, confirmed

The real Hitters correlation matrix - exactly the pattern we predicted:



Pair	Correlation
CAtBat ↔ CHits	1.00
CAtBat ↔ CRuns	0.98
CHits ↔ CRuns	0.98
CAtBat ↔ CRBI	0.95

This is why Lasso zeroed them out:

- CAtBat and CHits are essentially interchangeable with CRuns (correlation ≥ 0.98)
- once CRuns is in the model (coef. 151.85), the other two add almost no new information,
- so Lasso drops them to exactly zero.

VIF - the full multicollinearity picture

Variance Inflation Factor *for all 19 predictors, sorted from worst to best:*

Variable	VIF
CHits	502.95
CAtBat	251.56
CRuns	162.52
CRBI	131.97
CHmRun	46.49
Hits	30.28
AtBat	22.94
CWalks	19.74
Runs	15.25
RBI	11.92

Variable	VIF
Years	9.33
HmRun	7.76
Walks	4.15
League_N	4.13
NewLeague_N	4.10
Assists	2.71
Errors	2.21
PutOuts	1.24
Division_W	1.08

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

X_vif = X.copy()  # as mesmas 19 variáveis, antes do train_test_split

vif_data = pd.DataFrame()
vif_data["Variable"] = X_vif.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i)
                   for i in range(X_vif.shape[1])]

print(vif_data.sort_values("VIF", ascending=False))
```

Three tiers, one clean story:

- A severe tier (CHits, CAtBat, CRuns, CRBI, all VIF > 130): Lasso zeroed exactly the two worst (CHits, CAtBat), keeping CRuns/CRBI.
- A moderate tier (VIF 11- 46, including Hits & AtBat: surviving the Lasso isn't proof of independence).
- A clean tier (VIF < 10) defensive & categorical variables, genuinely unrelated to batting stats.

Coefficient by coefficient: Ridge vs. Lasso

All 19 predictors, both models (λ=4 for Ridge, CV-chosen λ for Lasso):

Variable	Ridge	Lasso
AtBat	-206.43	-234.21
Hits	217.58	277.64
HmRun	-24.60	**0.00**
Runs	19.46	**0.00**
RBI	25.95	**0.00**
Walks	86.38	85.07
Years	-36.51	-29.32
CAtBat	-110.75	**0.00**
CHits	119.98	**0.00**
CHmRun	95.85	85.04

Variable	Ridge	Lasso
CRuns	142.90	151.85
CRBI	94.61	89.00
CWalks	-105.38	-97.91
PutOuts	84.56	83.94
Assists	-0.27	**0.00**
Errors	9.99	8.87
League_N	18.61	14.69
Division_W	-59.28	-56.97
NewLeague_N	3.30	5.28

The real story:

- only 6 of 19 are zeroed;
- two separate redundant families: HmRun/Runs/RBI (already captured by AtBat/Hits) and CAtBat/CHits (already captured by CRuns, corr. ≥ 0.98).
- The other 13 (including AtBat, Hits, Walks, Years) carry genuinely distinct information, so they survive in both models.

SUMMARY

Ridge vs. Lasso: the final scoreboard

Model	Test MSE	Nonzero coefficients
OLS (λ=0)	133,595.0	19 / 19
Ridge (λ=4)	126,497.6	19 / 19
RidgeCV (λ=2.32)	126,176.9	19 / 19
LassoCV	124,887.9	13 / 19

Lasso wins on both counts here: lowest test MSE, and a simpler, sparser model. This won't always happen, but it shows why Lasso is often preferred when interpretability matters.

Elastic Net: the best of both?

Linearly combines the L1 (Lasso) and L2 (Ridge) penalties:

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2=RSS+\lambda\sum_{j=1}^{p}\beta_j^2$$

$$RSS+\lambda\sum_{j=1}^{p}\left[(1-\alpha)\tfrac{1}{2}\beta_j^2+\alpha|\beta_j|\right]$$

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}|\beta_j|=RSS+\lambda\sum_{j=1}^{p}|\beta_j|$$

where $\lambda \geq 0$ is a **tuning parameter** and $0 \leq \alpha \leq 1$ is the **mixing parameter**, both to be determined separately.

Alpha (sklearn: l1_ratio)	Behaviour
0	Pure Ridge (only L2)
1	Pure Lasso (only L1)
0.5	Equal 50/50 mix of both penalties

*Ridge, Lasso and Elastic Net can all be fit efficiently along a sequence of λ values. It's known as a **regularization path**.*

*In Elastic Net there are **two separate parameters controlling different things**:*

- α (alpha), l1_ratio: controls what kind of penalty is used: 0 = pure L2 (Ridge only), 1 = pure L1 (Lasso only), 0.5 = a 50/50 mix.

- λ(lambda): controls how much penalty is applied overall. The larger it is, the more aggressively the model shrinks (and potentially zeroes out) the coefficients.

Fitting Elastic Net on Hitters

PYTHON

```
from sklearn.linear_model import ElasticNet

# l1_ratio: 0=Ridge, 1=Lasso, 0.5=50/50 mix
elastic_net = ElasticNet(
    alpha=1.0, l1_ratio=0.5,
    random_state=42, max_iter=10000)

elastic_net.fit(X_train, y_train)
y_pred = elastic_net.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

TEST MSE

133,652.8

R^2

0.416

With these untuned defaults (α=1, l1_ratio=0.5), Elastic Net lands close to plain OLS
- a reminder that α and l1_ratio both still need tuning, e.g. via ElasticNetCV.

[ElasticNet — scikit-learn 1.9.1 documentation]

Coefficient by coefficient: Ridge vs. Lasso vs. Elastic Net

All 19 predictors, all CV-tuned (Ridge λ=4, Lasso & Elastic Net by CV; EN l1_ratio=0.99, α=0.469):

Variable	Ridge	Lasso	Elastic Net
AtBat	-206.43	-234.21	-273.5
Hits	217.58	277.64	289.47
HmRun	-24.60	**0.00**	-10.26
Runs	19.46	**0.00**	-0.00
RBI	25.95	**0.00**	13.59
Walks	86.38	85.07	99.97
Years	-36.51	-29.32	-20.91
CAtBat	-110.75	**0.00**	**-254.77**
CHits	119.98	**0.00**	**178.35**
CHmRun	95.85	85.04	97.50

Variable	Ridge	Lasso	Elastic Net
CRuns	142.90	151.85	227.00
CRBI	94.61	89.00	107.55
CWalks	-105.38	-97.91	-133.52
PutOuts	84.56	83.94	85.39
Assists	-0.27	**0.00**	7.34
Errors	9.99	8.87	12.03
League_N	18.61	14.69	20.40
Division_W	-59.28	-56.97	-57.87
NewLeague_N	3.30	5.28	1.78

A genuine surprise: Elastic Net's CV chosen l1_ratio=0.99 is almost pure Lasso

- yet it zeroes only 1 variable (Runs), not 6.
- Its chosen α (0.469) is far smaller than Lasso's, so it shrinks far less overall: l1_ratio alone doesn't determine sparsity, α matters just as much.

PRACTICAL GUIDANCE

So, which one should I use?

Many predictors **genuinely matter**, **somewhat correlated**	**Ridge**
Expect many coefficients to be **truly zero**; **want a sparse, interpretable model**	**Lasso**
Predictors are highly correlated in groups (Lasso picks one arbitrarily)	**Elastic Net**
Not sure, or p is huge relative to n	**Elastic Net**

SUMMARY

All the methods, side by side

Method	Penalty added to RSS	Sparse?	Tuning parameter(s)
OLS	RSS	No	n/a
Ridge	RSS + $\lambda\Sigma\beta_j^2$	No	λ
Lasso	RSS + $\lambda\Sigma\lvert\beta_j\rvert$	Yes	λ
Elastic Net	RSS + $\lambda[(1-\alpha)\frac{1}{2}\Sigma\beta_j^2 + \alpha\Sigma\lvert\beta_j\rvert]$	Yes	λ, α (l1_ratio)

- All three are fit by adding a penalty to RSS.
- The only real differences are the shape of that penalty, and whether it can zero out coefficients.

KEY TAKEAWAYS

Wrapping up

- Regularization fits all *p* predictors, but shrinks coefficients towards zero to reduce variance
- Ridge (L2 penalty) shrinks smoothly: never exactly to zero; handles multicollinearity well
- Lasso (L1 penalty) can zero out coefficients: built-in variable selection, sparser models
- On Hitters, both beat plain OLS; Lasso edged out Ridge here, with a far simpler model (13 vs. 19 predictors)
- Elastic Net blends both penalties: useful with correlated predictor groups; λ (and l1_ratio) should always be chosen by cross-validation

Regularization for Classification

Ridge, Lasso, Elastic Net to **Logistic Regression**.

Same idea, different loss function

Regularization always adds a penalty to a loss function (only the base loss changes):

Model	Base loss	Where
Linear Regression	$\text{RSS} = \Sigma(y_i - \hat{y}_i)^2$	(what we just used, Hitters)
Logistic Regression	– log-likelihood of the logistic model	(same idea, classification)

What stays exactly the same: adding $\lambda\Sigma\beta_j^2$ (Ridge), $\lambda\Sigma|\beta_j|$ (Lasso), or a mix of both (Elastic Net) to whichever loss you're minimizing. Shrinkage, sparsity, the bias-variance trade-off - all of it carries over unchanged.

The three penalties, for Logistic Regression

Ridge $-\log\text{-likelihood} + \lambda\Sigma\beta_j^2$

Lasso $-\log\text{-likelihood} + \lambda\Sigma|\beta_j|$

Elastic Net $-\log\text{-likelihood} + \lambda[(1-\alpha)\tfrac{1}{2}\Sigma\beta_j^2 + \alpha\Sigma|\beta_j|]$

Same shrinkage behaviour as before: Ridge never zeroes, Lasso can, Elastic Net blends the two.

Scikit-learn: parameters naming

Class	Parameter	Direction
`Ridge() / Lasso()  (regression)`	**alpha = λ**	larger alpha → more shrinkage
`LogisticRegression()  (classification)`	***C* = 1/λ**	smaller *C* → more shrinkage

This catches people out:

- in Ridge/Lasso, bigger alpha = stronger penalty.
- In LogisticRegression, bigger *C* = weaker penalty (the direction flips).

The same dataset, now as classification

New target: HighSalary (above/below median), nicely balanced (134 vs. 129 examples) :

PYTHON

```
from sklearn.linear_model import LogisticRegression

configs = [
    ("Ridge",       dict(l1_ratio=0.0, C=1.0)),
    ("Lasso",       dict(l1_ratio=1.0, C=1.0)),
    ("Elastic Net", dict(l1_ratio=0.5, C=1.0)),
]

results = {}
for name, params in configs:
    clf = LogisticRegression(solver='saga', max_iter=10000, **params)
    clf.fit(X_train, y_train)
    acc = accuracy_score(y_test, clf.predict(X_test))
    nz = np.sum(np.abs(clf.coef_[0]) > 1e-4)
    results[name] = clf.coef_[0]
    print(f"{name}: accuracy={acc:.4f}, nonzero={nz}/{X.shape[1]}")

print("\nCoeficientes:")
print(pd.DataFrame(results, index=X.columns).round(3))
```

Penalty	Accuracy	Nonzero
Ridge	0.9057	19 / 19
Lasso	0.8868	10 / 19
Elastic Net	0.9057	16 / 19

Same accuracy here, but Lasso again uses noticeably fewer features: sparsity, not just accuracy, is often the real reason to prefer it.

The multicollinearity story repeats: CAtBat and CHits are zeroed again (0.000) — and CRuns survives with an even LARGER coefficient (1.903) than in Ridge (1.199), exactly like it did in the regression case. Same predictors, same redundancy, now in a classification task.

[Lasso, Lasso-LARS, and Elastic Net paths — scikit-learn 1.9.1 documentation]

Regression vs. Classification: what gets zeroed

Same 19 predictors, same dataset - which ones does Lasso keep, in each task?

Variable	Regression (Salary)	Classification (HighSalary)
CAtBat	0.00 (zeroed)	0.00 (zeroed)
CHits	0.00 (zeroed)	0.00 (zeroed)
CRuns	151.85 (kept)	1.903 (kept)
CRBI	89.00 (kept)	0.00 (zeroed)
CHmRun	85.04 (kept)	0.00 (zeroed)
CWalks	-97.91 (kept)	-0.334 (kept)

The constant:

- CAtBat and CHits are always redundant with CRuns (corr. ≥ 0.98), **zeroed in both tasks**.

What changes:

- CRBI and CHmRun survive in regression but get zeroed in classification;
- the exact sparsity pattern depends on the task, not just the predictors' correlations.

KEY TAKEAWAYS

Regularization isn't just for regression

- Ridge, Lasso and Elastic Net generalize directly to Logistic Regression (only the base loss changes, from RSS to −log-likelihood)
- Shrinkage, sparsity and the bias-variance trade-off all carry over unchanged
- Watch the sklearn gotcha: LogisticRegression uses $C = 1/\lambda$ (smaller C means MORE regularization, the opposite of alpha in Ridge/Lasso)
- Try this directly on the Default dataset from our Classification lecture (same ideas, a dataset you already know)

REFERENCES

Further reading

James, Witten, Hastie & Tibshirani — An Introduction to Statistical Learning with Python (ISLP), Chapter 6: Linear Model Selection and Regularization. (`https://hastie.su.domains/ISLP/ISLP_website.pdf`*)*

Statistical Learning: 6.6 Shrinkage methods and ridge regression – YouTube
Statistical Learning: 6.7 The Lasso – YouTube
Statistical Learning: 6.8 Tuning parameter selection - YouTube

Regularization Part 1: Ridge (L2) Regression – YouTube
Regularization Part 2: Lasso (L1) Regression – YouTube
Ridge vs Lasso Regression, Visualized!!! – YouTube
Regularization Part 3: Elastic Net Regression - YouTube